DevBoard

Site de production : https://ece-webtech-302.vercel.app

Dépôt GitHub : https://github.com/ivanfocsa/ece-webtech-302

Technologies : Next.js 15, PostgreSQL (Supabase), Tailwind 4, Vercel

Groupe (3 étudiants) : Yvan Focsa, Kardiatou Touré, Emir Ben Hassen

DÉROULÉ DE LA SOUTENANCE (10-12 MINUTES AU TOTAL)

1. Yvan FOCSA

Lead, Architecture, Authentification & Déploiement (~3 min 30)

- **Présentation globale :** Expliquer le concept de DevBoard (plateforme de partage d'articles et veille tech réalisée avec Next.js 15).
- **Architecture Next.js 15 App Router :** Justifier le découpage entre Server Components (RSC) pour le rendu initial rapide sans JavaScript superflu et Client Components (RCC) pour l'interactivité.
- **Double flux d'authentification :** Démontrer la connexion GitHub OAuth 2.0 (échange sécurisé de code via PKCE) et l'alternative classique par email et mot de passe natif avec Supabase Auth.
- **Sécurité des sessions et middleware :** Détailler `middleware.js` qui intercepte chaque requête, lit les cookies chiffrés avec `@supabase/ssr` et redirige automatiquement vers `/login?next=...` sur les routes protégées (`/profile`, `/articles/new`, édition).
- **Déploiement Vercel :** Pipeline CI/CD connecté au dépôt GitHub, gestion des variables d'environnement et revalidation automatique.

2. Kardiatou TOURE

Backend, Base de données & Sécurité RLS (~3 min 30)

- **Modélisation PostgreSQL :** Présenter le schéma relationnel à 4 tables (`articles`, `comments`, `likes`, `contacts`), avec clés étrangères et suppression en cascade (`ON DELETE CASCADE`).
- **Sécurité par les données (Row Level Security) :** Expliquer que la sécurité ne repose pas seulement sur l'interface React, mais sur le moteur PostgreSQL :
  - Lecture publique des articles et commentaires pour tous les visiteurs (`using (true)`).
  - Création, modification et suppression strictement réservées à l'auteur de l'article (`using (auth.uid() = user_id)`). Même via Postman ou curl, personne ne peut altérer l'article d'un autre.
  - Ajout de likes et de commentaires restreint aux utilisateurs connectés.
  - Contrainte d'unicité composite `(article_id, user_id)` pour interdire les likes en double.
  - Insertion libre pour le formulaire de contact mais lecture restreinte.
- **Requêtes directes :** Code SQL propre et direct dans `lib/articles.js`, sans ORM lourd, pour une lisibilité maximale à l'oral.

3. Emir Ben Hassen

Frontend, Design Tailwind & Composants Interactifs (~3 min 30)

- **Design épuré monochrome :** Justifier le choix d'un thème sobre en noir et blanc réalisé avec Tailwind CSS 4 (moteur Oxide rapide, sans configuration complexe).
- **Composants réactifs (CSR) :** Bouton de like avec mise à jour optimiste, fil de commentaires et formulaire de contact interactif.
- **API météo externe :** Intégration de l'API publique Open-Meteo pour afficher en temps réel la température et la météo de Paris sur l'accueil, sans clé d'API requise.
- **Ergonomie mobile :** Interface 100% responsive, menu de navigation rétractable (hamburger) et navigation au clavier accessible.

LES 5 STRATÉGIES DE RENDU NEXT.JS (DÉMONSTRATION BUILD)

Stratégie	Route	Symbole Build	Fonctionnement concret dans DevBoard
SSG (Statique)	`/about`	○ (Static)	Page d'information pré-rendue à 100% au build. Fichier HTML servi directement par le CDN sans calcul serveur ni dépendance dynamique.
SSG Dynamique	`/articles/[id]`	● (SSG)	Utilise `generateStaticParams()` pour lire la base PostgreSQL au build et pré-générer les fiches d'articles existantes pour un chargement instantané.
SSR (Serveur)	`/articles` `/profile`	ƒ (Dynamic)	Rendu dynamique à chaque requête. `/articles` analyse les paramètres `searchParams` (recherche et filtre). `/profile` lit les cookies de session utilisateur.
ISR (Incrémental)	`/`	○ (ISR `60s`)	Page d'accueil régénérée en arrière-plan toutes les 60 secondes grâce à `export const revalidate = 60`, garantissant des articles récents et la météo à jour.
CSR (Client)	`/contact` `/login` `/articles/new`	○ (Client Shell)	Composants `'use client'` gérant les formulaires, la validation en direct et les appels asynchrones vers Supabase Auth et la base de données.

QUESTIONS PROBABLES DU PROFESSEUR & RÉPONSES TYPES

Pourquoi avoir choisi Supabase plutôt qu'une base SQLite ou MongoDB ?
Supabase fournit PostgreSQL, une vraie base relationnelle, avec l'authentification gérée et surtout le Row Level Security (RLS) directement exécuté par le moteur SQL. C'est plus sécurisé et adapté au travail en équipe.

Comment garantissez-vous qu'un utilisateur ne peut pas modifier l'article d'un autre ?
Par la politique SQL `using (auth.uid() = user_id)` sur la table articles. Même si un utilisateur malveillant envoie une requête HTTP directe en contournant l'interface React, le moteur PostgreSQL refuse l'écriture.

À quoi sert generateStaticParams sur les articles ?
Cette fonction extrait les identifiants d'articles au moment du build pour que Next.js génère les pages HTML à l'avance. Le temps d'affichage pour le lecteur est quasi immédiat et le SEO est optimal.

Où vont les messages envoyés depuis le formulaire de contact ?
Ils sont insérés directement dans la table `contacts` de PostgreSQL. La politique RLS autorise l'insertion publique, et les messages sont consultables dans le Table Editor de Supabase.

Comment fonctionne le flux OAuth 2.0 avec GitHub ?
L'utilisateur est redirigé vers GitHub pour autoriser l'accès à son profil. GitHub renvoie un code temporaire vers notre callback Supabase, qui l'échange contre un jeton de session stocké dans un cookie HTTP chiffré.

Pourquoi le choix de Tailwind CSS 4 ?
Tailwind 4 intègre le nouveau moteur Oxide en Rust. Il fonctionne nativement en CSS moderne sans avoir besoin d'un gros fichier de configuration `tailwind.config.js`, ce qui accélère le build.